

Identifica el ciclo de los componentes

Clase 5

```
constructor() {  
  super()  
  this.title = React.createRef()  
}  
render() {  
  return (  
    <div className="App">  
      <h1 ref = { this.title }>Hola comunidad</h1>  
    </div>  
  );  
}
```

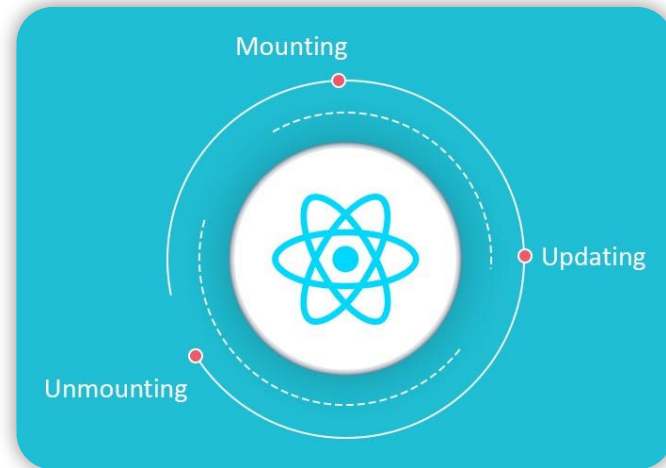




Ciclos de los componentes

En React, los componentes tienen un **conjunto de métodos de ciclo de vida** que se ejecutan en diferentes etapas, como montaje, actualización y desmontaje. Puedes implementar estos métodos en tus componentes para realizar acciones específicas en cada etapa del ciclo de vida.

Por ejemplo, puedes usar el método **componentDidMount()** para realizar tareas después de que el componente se monte en el DOM, como hacer una solicitud HTTP para cargar datos. Del mismo modo, puedes usar **componentDidUpdate()** para realizar acciones después de que el componente se actualice.

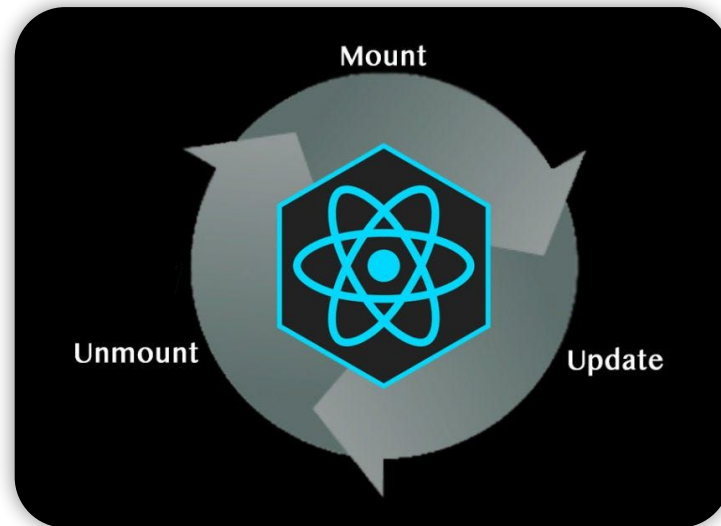




Etapas del Ciclo de los componentes

En React, los componentes tienen un ciclo de vida que consta de diferentes etapas, cada una con métodos específicos que se ejecutan en momentos clave durante la existencia del componente. Estas etapas incluyen:

- 1) **Mounting**
- 2) **Updating**
- 3) **Unmounting**





Métodos de los ciclos de componentes

Métodos de Montaje (Mounting)

- **constructor():** Este es el primer método que se llama cuando se crea una instancia del componente. Se utiliza para inicializar el estado del componente y enlazar métodos de instancia.
- **render():** Este método es obligatorio y se llama después del constructor. Retorna el JSX que representa el componente. En este método se define la estructura inicial de la interfaz de usuario.
- **componentDidMount():** Se llama después de que el componente se monta en el DOM. Es útil para realizar operaciones que requieren acceso al DOM o para realizar solicitudes de red para cargar datos iniciales.



Métodos de los ciclos de componentes

Métodos de Actualización (Updating)

- **static getDerivedStateFromProps()**: Este método estático se llama antes de renderizar cuando las nuevas propiedades (props) se reciben. Se utiliza para calcular el estado del componente basado en las nuevas propiedades y el estado actual.
- **shouldComponentUpdate()**: Se llama antes de que se realice la actualización del componente. Permite controlar si el componente debe volver a renderizarse o no. Puede optimizar el rendimiento evitando actualizaciones innecesarias.
- **render()**: Al igual que en el montaje, este método es obligatorio y se llama para representar el componente después de recibir nuevas props o estado.
- **getSnapshotBeforeUpdate()**: Se llama justo antes de que los cambios sean reflejados en el DOM. Permite capturar información del DOM antes de que sea potencialmente modificada.
- **componentDidUpdate()**: Se llama después de que se hayan aplicado las actualizaciones al DOM. Es útil para realizar operaciones de limpieza, como la actualización de datos o la interacción con el DOM.



Métodos de los ciclos de componentes

Métodos de Desmontaje (Unmounting)

- **componentWillUnmount():** Se llama justo antes de que el componente sea eliminado del DOM. Es el lugar adecuado para realizar tareas de limpieza, como la cancelación de peticiones de red, la eliminación de temporizadores o la limpieza de suscripciones.



Ejemplo de métodos de ciclo de componentes

```
import React, { Component } from 'react';

class MiComponente extends Component {
  constructor(props) {
    super(props);
    this.state = {
      mensaje: 'Hola, mundo!'
    };
    console.log('Constructor ejecutado');
  }

  componentDidMount() {
    console.log('El componente se ha montado en el DOM');
  }

  componentDidUpdate() {
    console.log('El componente se ha actualizado');
  }

  componentWillUnmount() {
    console.log('El componente se ha desmontado del DOM');
  }

  render() {
    console.log('El método render se ha ejecutado');
    return (
      <div>
        <h1>{this.state.mensaje}</h1>
        <button onClick={() => this.setState({ mensaje: '¡Hola, React!' })}>
          Cambiar Mensaje
        </button>
      </div>
    );
  }
}

export default MiComponente;
```

En este ejemplo....

Este componente **MiComponente** ilustra el uso de varios métodos del ciclo de vida de los componentes en React, como **constructor**, **componentDidMount**, **componentDidUpdate** y **componentWillUnmount**.

Si ejecutas este ejemplo, revisa la **consola**.



Ejemplo de métodos de ciclo de componentes

The image shows a web browser window displaying a simple application with the text "¡Hola, React!" and a button labeled "Cambiar Mensaje". Below the browser window, the VS Code editor shows the source code for the application. The code defines a class component `MiComponente` with the following lifecycle methods:

- `constructor(props)`: Initializes the component's state with `mensaje: 'Hola, mundo!'` and logs "Constructor ejecutado".
- `componentDidMount()`: Logs "El componente se ha montado en el DOM".
- `componentWillUnmount()`: Logs "El componente se ha desmontado del DOM".
- `componentDidUpdate()`: Logs "El componente se ha actualizado".
- `render()`: Returns the JSX for the component, which includes a `h1` tag with the state's `mensaje` and a button that calls `handleClick` to update the state.

The VS Code interface also shows the Explorer sidebar with the project structure, the Console with the output of the lifecycle methods, and the Debug Console with the message "webpack compiled successfully".

